

Département d'Informatique
Licence IA-option : Sciences du Logiciel
Module : Programmation Web Dynamique

Projet de Fin de Module

Accessoires

Développement d'une Boutique en Ligne d'accessoires

Réalisé par :

Radia BOUZINE

Iymane BOLAKHRIF

Mouna GUALY

À l'attention de :

Pr. Dounia LOTFI

Année Universitaire : 2025 – 2026

TABLE DES MATIÈRES :

1. Introduction.....	3
2. Analyse et Conception.....	4
3. Environnement de Développement.....	6
4. Base de données.....	7
5. Implémentation.....	9
6. Sécurité.....	13
7. Tests et Validation.....	14
8. Conclusion.....	15

1. Introduction

1.1 Contexte du projet

Ce projet s'inscrit dans le cadre du module de Programmation Web Dynamique de la Licence IA-option Sciences du Logiciel de l'Université Mohammed V, Faculté des Sciences de Rabat. Il vise à mettre en pratique les compétences acquises des concepts vus en cours : PHP orienté objet, architecture MVC, gestion de sessions, base de données MySQL.

1.2 Présentation du projet

Accessoires est une boutique en ligne spécialisée dans la vente d'accessoires. Le nom évoque directement l'univers des produits proposés : montres, bijoux, sacs, lunettes, ceintures, etc.

La plateforme propose une expérience d'achat complète, depuis la découverte des produits jusqu'à la validation de la commande, avec une interface élégante adaptée à l'image de marque haut de gamme visée.

1.3 Objectifs

- Développer une application web dynamique en PHP avec architecture MVC
- Permettre la gestion complète du cycle de vente : catalogue, panier, commandes
- Implémenter un système d'authentification sécurisé pour deux rôles (admin / client)
- Intégrer un assistant chatbot basé sur l'analyse de mots-clés
- Fournir un tableau de bord d'administration avec statistiques
- Garantir la sécurité contre les attaques courantes (XSS, CSRF, injections SQL)

2. Analyse et Conception

2.1 Analyse du Cahier des Charges

Le cahier des charges du projet définit une plateforme e-commerce articulée autour de deux types d'acteurs : l'administrateur et le client. L'analyse des besoins a permis d'identifier les fonctionnalités prioritaires à implémenter.

Acteur	Fonctionnalités
Administrateur	Gestion des produits (CRUD), gestion des commandes (statuts), gestion des comptes, tableau de bord statistiques
Client	Inscription / connexion, navigation catalogue, filtrage produits, panier, validation de commande, historique commandes
Visiteur	Consultation du catalogue, recherche produits, accès au chatbot

2.2 Architecture MVC

L'application est organisée selon le patron de conception MVC (Modèle-Vue-Contrôleur), qui favorise la séparation des responsabilités et facilite la maintenance du code. Cette architecture est implémentée à travers quatre fichiers principaux :

Couche	Fichier	Rôle
Modèle	modele.php	Accès base de données, logique métier, fonctions CRUD
Vue	template.php	Génération du HTML, rendu des pages et composants visuels
Contrôleur	controleur.php	Routage des requêtes, orchestration des appels modèle/vue
Point d'entrée	index.php	Initialisation de la session, inclusion des composants

Cette organisation garantit un code structuré et maintenable. Chaque composant peut être modifié indépendamment, ce qui facilite l'évolution et le débogage de l'application.

2.3 Modélisation de la Base de Données

La base de données MySQL comporte quatre tables interconnectées par des clés étrangères :

Table	Champs principaux	Description
utilisateurs	id, login, mdp (hashé), role	Comptes clients et administrateurs
produits	id, nom, prix, categorie, genre, description, image	Catalogue complet des accessoires
commandes	id, utilisateur_id, total, statut, mode_paiement	En-têtes de commandes avec suivi de statut
details_commandes	id, commande_id, produit_id, prix_unitaire, quantite	Lignes de détail de chaque commande

Les relations entre tables sont assurées par des contraintes de clés étrangères avec les règles ON DELETE CASCADE et ON DELETE SET NULL pour préserver l'intégrité référentielle des données.

3. Environnement de Développement

Le projet a été développé et testé dans un environnement local standard pour le développement web PHP, en suivant les recommandations du cahier des charges.

Composant	Technologie / Version
Serveur local	XAMPP (Apache + MariaDB 10.4.32)
Langage backend	PHP 8.2 avec déclaration strict_types
Base de données	MySQL / MariaDB avec PDO
Frontend	HTML5, CSS3, JavaScript (Vanilla)
Éditeur de code	Visual Studio Code
Gestion de version	Git
Administration BDD	phpMyAdmin 5.2.1

L'utilisation de PDO (PHP Data Objects) pour l'accès à la base de données offre plusieurs avantages : une couche d'abstraction permettant de changer de SGBD facilement, et surtout l'utilisation de requêtes préparées qui protègent contre les injections SQL.

4. Base de données

4.1 Présentation et description des tables

La base de données s'appelle : **boutique**

Elle contient quatre tables :

- ◆ **Table utilisateurs** : Stocke les informations de chaque utilisateur inscrit sur la plateforme.

utilisateurs (id, , login, mdp, role)

- ◆ **Table produits** : Contient les 93 produits du catalogue (montres, bijoux, sacs, lunettes, accessoires).

produits (id, nom, prix, categorie, description, image, genre)

- ◆ **Table commandes** : Enregistre chaque commande passée par un client.

commandes (id, utilisateur_id, nom_client, email_client, adresse, total, statut, en_attente, date_commande, mode_paiement, paiement_statut, paiement_reference, paiement_carte_last4)

- ◆ **Table details_commandes** : Table de liaison entre une commande et ses produits (relation N-N résolue).

details_commandes (id, commande_id, produit_id, nom_produit, prix_unitaire, quantité)

4.2 Relations entre tables

Les relations entre tables sont définies par des clés étrangères avec suppression en cascade :

- **utilisateurs (1) —> commandes (N)** : un client peut passer plusieurs commandes.
- **commandes (1) —> details_commandes (N)** : une commande contient plusieurs lignes.
- **produits (1) —> details_commandes (N)** : un produit peut figurer dans plusieurs commandes.

5. Implémentation

5.1 Structure du Projet

Le projet est organisé de façon claire avec une séparation nette entre les fichiers PHP, les ressources statiques et les images produits :

Fichier / Répertoire	Contenu / Rôle
index.php	Point d'entrée unique, démarrage de session, inclusion MVC
controleur.php	Routage des pages (464 lignes), gestion des actions POST/GET
modele.php	Fonctions base de données et logique métier (641 lignes)
template.php	Templates HTML de toutes les pages (541 lignes)
config.php	Paramètres de connexion base de données
assets/css/style.css	Feuille de styles globale responsive
assets/js/app.js	Scripts JavaScript (panier dynamique, chatbot UI)
images/produits/	25 images de produits (JPEG)
assets/images/	Icônes SVG par catégorie (sacs, bijoux, montres...)

5.2 Gestion des Utilisateurs et Authentification

Le système d'authentification gère deux rôles distincts : client et administrateur. La création de compte utilise la fonction `password_hash()` de PHP avec l'algorithme `bcrypt`, garantissant un stockage sécurisé des mots de passe.

À la connexion, les informations de l'utilisateur sont stockées en session (identifiant, login, rôle). La vérification des droits d'accès est effectuée à chaque page sensible via la fonction `exigerAdminControleur()`, qui redirige vers la page de connexion en cas d'accès non autorisé.

Deux comptes sont pré-configurés en base de données pour les démonstrations : un compte administrateur (login : admin) et un compte client (login : client). Les mots de passe sont stockés sous forme de hachages SHA-256 et bcrypt respectivement.

5.3 Catalogue Produits et Filtrage

Le catalogue contient 25 produits répartis en 5 catégories : sacs, bijoux, montres, lunettes et ceintures. Chaque produit possède un attribut genre (homme, femme, mixte) permettant une navigation segmentée.

Le système de filtrage implémenté dans la fonction `getProduits()` du modèle permet de combiner plusieurs critères simultanément :

- Filtrage par catégorie (sacs, bijoux, montres, lunettes, ceintures)
- Filtrage par genre (homme, femme, mixte)
- Filtrage par fourchette de prix (prix_min, prix_max)
- Recherche textuelle sur le nom et la description du produit
- Tri par nom, prix croissant ou prix décroissant

Les requêtes SQL sont construites dynamiquement avec des paramètres liés (prepared statements) pour garantir la sécurité et les performances.

5.4 Gestion du Panier et des Commandes

Le panier est géré côté serveur via les sessions PHP, ce qui le rend accessible sans base de données et sans nécessiter de connexion préalable. La structure du panier en session est un tableau associatif de type `[produit_id => quantite]`.

Le processus de commande se déroule en plusieurs étapes :

- Le client ajoute des articles au panier depuis les fiches produits

- Il accède à la page panier pour vérifier et modifier les quantités
- Il remplit le formulaire de commande (nom, email, adresse, mode de paiement)
- La commande est enregistrée en base de données avec ses lignes de détail
- Un email de confirmation est simulé et une page de confirmation s'affiche

Deux modes de paiement sont supportés : paiement à la livraison (cash) et paiement par carte bancaire. Pour ce dernier, une validation de l'algorithme de Luhn est implémentée côté serveur pour vérifier la validité formelle du numéro de carte.

5.5 Chatbot d'Assistance

Un assistant conversationnel est intégré dans l'interface sous forme de widget flottant accessible depuis toutes les pages. Son fonctionnement repose sur l'analyse de mots-clés dans les questions posées par l'utilisateur.

La fonction `chatbotReponse()` du modèle implémente plusieurs scénarios de réponse :

- Identification d'une catégorie de produit : retourne les 3 articles les moins chers de cette catégorie
- Détection d'un budget : retourne les produits dont le prix est inférieur au montant mentionné
- Détection d'une intention d'achat cadeau : retourne 3 produits aléatoires en suggestion
- Questions sur la livraison ou le suivi : redirige vers l'espace client
- Recherche textuelle générale : interroge le catalogue sur le nom et la description

La communication entre le frontend (JavaScript) et le chatbot se fait via une requête AJAX en POST, qui retourne une réponse JSON. Cette approche évite le rechargement de la page et offre une expérience fluide.

5.6 Interface d'Administration

Le tableau de bord administrateur donne accès à trois sections principales :

- Statistiques globales : nombre total de produits, de commandes et chiffre d'affaires cumulé
- Gestion des produits : ajout, modification et suppression avec upload d'images
- Gestion des commandes : consultation des détails et mise à jour du statut (en attente, confirmée, expédiée, livrée, annulée)

L'accès à toutes les pages d'administration est protégé par la vérification du rôle admin en session. Toute tentative d'accès non autorisé est interceptée et redirigée vers la page de connexion avec un message d'erreur.

6. Sécurité

La sécurité de l'application a été une priorité tout au long du développement. Plusieurs mécanismes ont été mis en place pour protéger l'application et les données des utilisateurs.

Menace	Mesure de protection implémentée
Injection SQL	Utilisation systématique de PDO avec requêtes préparées et <code>bindValue()</code>
Cross-Site Scripting (XSS)	Échappement de toutes les sorties HTML via la fonction <code>h()</code> (<code>htmlspecialchars()</code>)
Cross-Site Request Forgery (CSRF)	Token CSRF généré par session, inclus dans tous les formulaires et vérifié à la soumission
Accès non autorisé	Vérification du rôle en session avant chaque action sensible
Mots de passe faibles	Hachage bcrypt via <code>password_hash()</code> / <code>password_verify()</code>
Validation des données	Validation côté serveur de toutes les entrées (types, longueurs, formats)

La déclaration `declare(strict_types=1)` en tête des fichiers PHP garantit également un typage strict, réduisant les risques liés aux conversions implicites de types qui peuvent parfois ouvrir des failles de sécurité.

7. Tests et validation

L'application a été testée de manière fonctionnelle en simulant les principaux scénarios d'utilisation. Les tests ont couvert les flux suivants :

Scénario testé	Résultat	Statut
Inscription d'un nouveau client	Compte créé, session ouverte	Validé
Connexion admin / client	Redirection correcte selon rôle	Validé
Ajout / modification produit (admin)	Persistance en base de données	Validé
Filtrage multi-critères produits	Résultats cohérents	Validé
Ajout au panier et modification quantité	Total recalculé en temps réel	Validé
Validation commande (cash et carte)	Enregistrement en base, panier vidé	Validé
Mise à jour statut commande (admin)	Statut mis à jour en base	Validé
Chatbot : requête catégorie et budget	Réponses pertinentes retournées	Validé
Tentative d'accès admin sans droits	Redirection vers login	Validé

Les tests de sécurité ont également été réalisés manuellement en tentant des injections SQL dans les champs de recherche et des tentatives de XSS dans les formulaires, sans succès grâce aux mesures de protection mises en place.

9. Conclusion

Ce projet a permis de développer une application web de vente d'accessoires en utilisant PHP, MySQL et l'architecture MVC. Les différentes fonctionnalités prévues ont été réalisées, notamment la gestion des utilisateurs, du catalogue de produits, du panier, des commandes et de l'administration. Une attention particulière a également été portée à la sécurité et à la validation des données. La réalisation de ce projet nous a permis de mettre en pratique les connaissances acquises durant le module de Programmation Web Dynamique et d'améliorer nos compétences en développement web, en conception de bases de données et en travail de projet.